1. We generate one response for each of the questions in the benchmark using the base model and also the base model with an appended obfuscating soft prompt at the end of the user message.
2. To avoid parsing problems coming from the model responding in a markdown style, or being forced to output more tokens than necessary, we (1) remove all ``` (triple back-ticks) from the generations and (2) split the generations to keep all of the text before the first `;` (semicolon, standard SQL query delimiter).
3. Run a script from the benchmarking suite on all of the resulting answers that runs each output query on a real database and checks the result against a ground truth.
4. Report the percentage of correctly answered queries among the 210 responses.

C.4 Additional results

Table 2 shows the confidence intervals for the Pearson correlation coefficient between number of layers obfuscated against and monitor recall or model SQLEval performance corresponding to results in Fig. 15.

Table 2: 95% confidence intervals for the Pearson correlation coefficient between number of layers obfuscated against and monitor recall or model SQLEval performance corresponding to results in Fig. 15. Bolded confidence intervals *do not* contain 0, thus indicating a statistically significant correlation.

Metric	SAE (raw)	SAE + LogReg	LogReg	MLP
Recall @1% FPR	$[-0.253, 0.333]$	**[0.343, 0.710]**	[-0.437, 0.334]	[-0.282, 0.410]
Performance on SQLEval	**[-0.879, -0.765]**	**[-0.916, -0.801]**	**[-0.881, -0.697]**	**[-0.820, -0.574]**

D Details on frontiers of obfuscated activations

D.1 Obfuscated Adversarial Training

In Algorithm 1 we provide the algorithm for Obfuscated Adversarial Training. In Fig. 22 we show the performance of generation and instruction (input) based harmfulness probes during the training of an OAT model.

Hyperparameter configuration. We use an effective batch size of 16. The probe and LoRA adapter learning rates are set to 10^{-3} and 10^{-4} respectively. The probe is trained for 1024 initial steps before being frozen, followed by 2048 steps of adversarial training where we initialize and train only the LoRA adapter. At every adversarial training step, the embedding suffix is trained for 32 iterations at a learning rate of 10^{-3}. We use Adam for the probe, adapter, and adversary. Robustness against universal embedding suffix is evaluated after every 128 steps of adversarial training.

Input: Model M, Randomly initialized monitor D, Harmful and benign training data, Learning rates α, β, Loss scaling term λ_{KL}.
Output: Model M with robust representations

Train monitor D to distinguish harmful and benign activations

for *batch* (X, y) *with harmful and benign data* **do**
  $W \leftarrow$ random per-example embedding suffixes `// Initialize attack suffixes`
  $X_{\text{adv}} := X_{\text{harm}} \oplus W$
  for *k steps* **do**
    $W \leftarrow W - \alpha \nabla_W (\mathcal{L}_{\text{task}}(M(X_{\text{adv}})) + D(\mathcal{R}(X_{\text{adv}})))$ `// Obfuscation attack`
  $\mathcal{L}_{\text{KL}} \leftarrow D_{KL}(M(X_{\text{benign}}) \parallel M_{\text{original}}(X_{\text{benign}}))$
  $\mathcal{L}_{\text{robust}} \leftarrow \text{BCE}(D(\mathcal{R}(X_{\text{benign}})), y_{\text{benign}}) + \text{BCE}(D(\mathcal{R}(X_{\text{adv}})), y_{\text{harm}})$
  $w_M \leftarrow w_M - \beta \nabla_{w_M} (\mathcal{L}_{\text{robust}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}})$ `// Update model weights` w_M `to make` D `robust against` X_{adv}

Algorithm 1: Obfuscated Adversarial Training (OAT). On each batch of data, we first run an obfuscation attack to find per-input embedding suffixes that jailbreak the model M while fooling the frozen monitor D. Then, we update the weights of M to lower the monitor's loss on a mix of benign data and the data produced via the obfuscation attack.

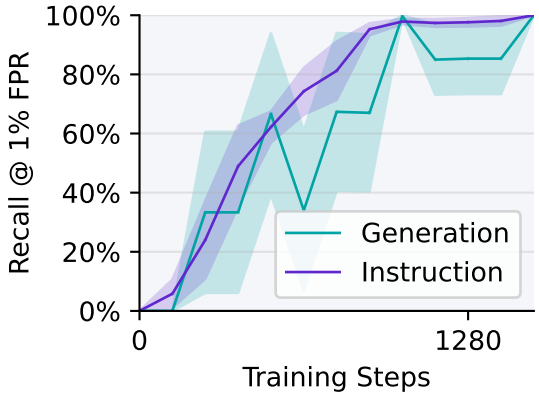

Figure 22: Recall at 1% FPR over training steps for probes during Obfuscated Adversarial Training (OAT) on LLaMa 3 8B. Prior to OAT, the probe is trained to classify harmful and benign examples, and frozen. We evaluate the probe after every 128 gradient steps of OAT, after attacking it with a new embedding suffix.

E Details on OOD detectors

Gaussian detector. We fit a multivariate Gaussian to the residual stream activations. We treat different layers as independent, fitting a separate Gaussian to each one. For generation-based detection, we also treat different tokens as independent, sharing a single Gaussian across all tokens.

To fit this Gaussian, we keep a running estimate of the mean and covariance matrix, which we

update on every batch. This method is exact, i.e., after a pass over the entire dataset, this running estimate is exactly the sample mean and sample covariance matrix of the activations on that dataset.

In some cases, our covariance estimate is based on fewer samples than the residual stream dimension, so a naive estimate would lead to a singular covariance matrix. Following Talur et al. [2023], we thus use a shrinkage estimator, i.e., we bias the covariance matrix estimate toward a multiple of the identity matrix.

To compute OOD scores, we use the squared Mahalanobis distance normalized by the residual stream dimension, i.e.,

$$D(\mathcal{R}(x)) = \frac{1}{d_{\text{res}}}(\mathcal{R}(x) - \mu)^T \Sigma_\alpha^{-1} (\mathcal{R}(x) - \mu)\,, \tag{10}$$

where d_{res} is the residual stream dimension, μ is the d_{res}-dimensional sample mean, and Σ_α is the $d_{\text{res}} \times d_{\text{res}}$ estimate of the covariance matrix using the shrinkage estimator. The normalization would matter when aggregating scores across activations with different dimensions, but since our experiments use only residual stream dimensions, it has no effect on metrics such as recall or FPR.

Variational Autoencoder (VAE). We use a VAE consisting of MLPs for the encoder and decoder. As for the Gaussian detector, we treat different layers and tokens separately, sharing a VAE across tokens and learning a separate one for each layer we use. We use ELBO as the OOD score assigned by the VAE detector.

F Why recall at low FPR?

When evaluating binary classifiers, we need to either decide on some approach for picking a decision threshold, or use a threshold-less metric such as AUROC. In this paper, we pick the threshold that leads to a false positive rate of 1% and then evaluate recall, and this appendix briefly explains that choice.

First, note that picking a threshold that achieves a 1% FPR is something a realistic defender can do, since the defender has access to benign (i.e., negative) samples in all our experiments. They can simply compute monitor scores on this benign data and pick a decision threshold such that around 1% of those scores are above the threshold. In contrast, a defender might not be able to target a specific recall (i.e., false negative rate), since they might not have access to samples from the attack distribution (e.g., inputs containing the backdoor trigger).

Why focus on a relatively low FPR like 1% specifically? In many monitoring settings, FPRs much higher than this would make a monitor unusable. For example, a harmfulness monitor that rejects 10% of benign user requests would likely not be deployed in most production use cases. (In fact, an even lower FPR will often be required in practice.)

Finally, why not use AUROC to avoid picking a threshold at all? AUROC essentially averages recall across all FPR values. This is meant to evaluate a classifier across a wide range of trade-offs between false negative rate and false positive rate. But as we argued, high FPRs are ultimately not acceptable for most of our target applications, and so the recall at those high FPRs is not very important for our purposes. An average across all FPRs, like AUROC, thus makes it hard to interpret performance in the relevant low-FPR regime. 95% AUROC might sound like a strong classifier, but it could easily be useless if an FPR of 1% is required.